

A PROJECT PROPOSAL

ON

FILE COMPRESSION SYSTEM USING HUFFMAN CODING

A Project Proposal Submitted in Partial Fulfillment of the
Requirements for the Degree of Bachelor of Technology

Submitted By:

Aarti Kushwaha

Under the Guidance of:

Project Supervisor

Academic Session 2025–2026

Certificate of Approval

This is to certify that the project proposal entitled “File Compression System Using Huffman Coding” submitted by Aarti Kushwaha in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology has been examined and approved as an authentic proposal work carried out under supervision.

Project Supervisor

Internal / External Examiner

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Background Study	1
1.3	Scope	2
1.4	Limitations	2
2	Problem Statement & Objectives	3
2.1	Problem Statement	3
2.2	Objectives	3
3	Methodology	5
3.1	Requirement Identification	5
3.1.1	Study of Existing System	5
3.1.2	Literature Review	5
3.1.3	Requirement Analysis	5
3.2	Feasibility Study	6
3.2.1	Technical Feasibility	6
3.2.2	Operational Feasibility	7
3.2.3	Economic Feasibility	7
3.3	High Level Design & Description of Algorithms	7
3.3.1	Working Mechanism of Proposed System	7
4	Project Timeline & Expected Outcomes	9
4.1	Gantt Chart (Project Timeline)	9
4.2	Expected Outcomes	9
5	Conclusion & References	11
5.1	Conclusion	11
5.2	References	11

Chapter 1

Introduction

1.1 Introduction

In modern computing environments, the rapid expansion of digital information has made efficient data transmission and compact storage critical necessities. Data compression algorithms minimize the physical space required to store information while conserving transmission bandwidth across communications networks.

Compression methods are fundamentally categorized into lossy and lossless algorithms. While lossy compression achieves higher compression ratios by discarding perceptual redundancies (such as in audio or photographic formats), lossless compression reconstructs original binary data exactly, bit-for-bit. For text documents, source code, database dumps, and application binaries, lossless compression is strictly mandatory.

This project focuses on the study, design, and implementation of a File Compression System using the classical Huffman Coding algorithm. The application enables users to compress and decompress arbitrary files efficiently through an interactive web-based graphical interface with complete lossless integrity.

1.2 Background Study

The theoretical foundation of lossless compression originated in Claude Shannon's landmark 1948 paper, "A Mathematical Theory of Communication", which defined the concept of Shannon Entropy as the fundamental lower bound on the average number of bits required to encode symbol alphabets.

In 1952, David A. Huffman developed an optimal greedy method for constructing variable-length prefix-free codes based on known symbol occurrence frequencies. Huffman coding establishes an optimal prefix property: no assigned code string is a prefix of any other code string, allowing continuous and unambiguous sequential decoding without special delimiters.

In conventional computing environments, proprietary or complex compression

utilities often mask the underlying algorithmic mechanisms. This proposal aims to provide an accessible, self-contained implementation that bridges theoretical information theory with practical software engineering.

1.3 Scope

The scope of this project encompasses:

1. Development of an automated byte-frequency counter and min-heap binary tree generator in software.
2. Generation of variable-length prefix codes and bit-level bitstream packing for arbitrary file formats.
3. Embedding self-contained header metadata containing serialized tree structures to permit independent decompression.
4. Lossless decompression engine to restore encoded files byte-for-byte with SHA-256 verification.
5. A clean, responsive user interface allowing drag-and-drop file upload, compression tracking, and history management.

1.4 Limitations

The system operates under the following identified limitations:

1. Pre-compressed File Limitations: Files that already exhibit high Shannon entropy (such as JPEG images, MP4 videos, or previously compressed ZIP files) cannot be substantially compressed further and may slightly expand due to the overhead of the frequency dictionary header.
2. Static Huffman Tree Overhead: For very small input files (e.g., files under 100 bytes), the storage cost of transmitting the codebook frequency table in the header can exceed the space saved from bit-level compaction.
3. Single-Node Execution: Initial implementation is targeted for standalone desktop and local web server execution rather than distributed cloud clusters.

Chapter 2

Problem Statement & Objectives

2.1 Problem Statement

Every day, computer users and organizations handle vast volumes of documents, log files, configuration data, and scripts. Storing and transmitting these files in raw uncompressed formats consumes excessive hard drive space and overburdens limited network bandwidth.

Existing industrial compression software tools are frequently closed-source, bundle complex external proprietary dependencies, or function as opaque black boxes that do not expose the internal mechanics of entropy reduction. Furthermore, many educational environments lack simple, transparent, and reproducible platforms for demonstrating variable-length entropy coding directly on actual user files.

Therefore, there is an evident need for an efficient, transparent, and robust File Compression System centered on the Huffman Coding algorithm, capable of providing verified lossless file reconstruction through an accessible interface.

2.2 Objectives

The primary objectives of the proposed project are as follows:

1. **Lossless Compression:** To develop a fully functional Huffman compression engine capable of compressing arbitrary file formats without losing a single byte of data.
2. **Accurate Decompression:** To implement an inverse tree-traversal decompression engine that reads bitstreams and perfectly reconstructs original files.
3. **Optimal Prefix Encoding:** To utilize a min-heap priority queue for constructing prefix-free binary codes with optimal average codeword length.
4. **Integrity Verification:** To compute and verify cryptographic SHA-256 checksums before compression and after decompression to ensure total data fidelity.

5. User-Friendly Interface: To provide an intuitive web-based interface for drag-and-drop file processing, download management, and session activity tracking.

Chapter 3

Methodology

3.1 Requirement Identification

3.1.1 Study of Existing System

An analysis of existing compression utilities reveals that while standard general-purpose utilities (such as gzip, 7-Zip, or WinRAR) achieve high compression ratios, they combine multi-stage algorithms (such as LZ77 dictionary matching with sliding windows followed by dynamic Huffman trees). This multi-stage pipeline makes debugging and educational comprehension complex. A pure, robust implementation of canonical Huffman coding allows direct analysis of symbol probability distribution and prefix code generation without unnecessary complexity.

3.1.2 Literature Review

1. Shannon (1948): Formulated the mathematical foundation of communication and proved that the entropy $H(X) = -\sum P(x_i) \log_2 P(x_i)$ represents the absolute theoretical limit for lossless data compaction.
2. Huffman (1952): Published “A Method for the Construction of Minimum-Redundancy Codes”, introducing the bottom-up tree generation algorithm using priority queues that guarantees optimal prefix codes for given character probabilities in $O(n \log n)$ time.
3. Knuth (1985): Explored dynamic and static Huffman data structures, demonstrating memory-efficient bit-packing techniques and sequential binary tree traversal mechanisms for digital computers.

3.1.3 Requirement Analysis

Functional Requirements

- The system shall read input files in binary stream mode to support both ASCII text and arbitrary binary data.

- The system shall construct byte-frequency tables across 256 possible byte values (0x00 to 0xFF).
- The system shall build an optimal binary tree using a priority queue.
- The system shall generate variable-length bit codewords and write packed bitstreams.
- The system shall include self-contained decoding headers in compressed archives.
- The system shall decompress archives and restore files with matching checksums.

Non-Functional Requirements

- Reliability: 100% lossless reconstruction with zero data corruption.
- Usability: Clean, modern, and responsive user interface.
- Performance: Rapid execution for common documents and source files.

Hardware & Software Requirements

- Hardware Requirements: Minimum 2.0 GHz Dual-Core CPU, 4 GB RAM, 500 MB free hard disk space.
- Software Requirements: Windows 10/11 or Linux OS, Modern Web Browser (Chrome/Firefox/Edge), PHP 8.2+ runtime environment.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The system utilizes modern PHP for backend processing, HTML5/CSS3/JavaScript for frontend presentation, and SQLite/MySQL for file logging. PHP provides native bitwise operators, stream wrappers, and hash libraries. Modern browsers support high-performance drag-and-drop file APIs. Hence, the project is completely technically feasible.

3.2.2 Operational Feasibility

The proposed application requires no specialized client-side software installations. Users interact with the platform through any standard web browser. The workflows for compression and decompression are straightforward, requiring only simple file uploads. Therefore, operational feasibility is high.

3.2.3 Economic Feasibility

The entire software stack comprises free, open-source technologies (PHP, SQLite, standard web specifications). No commercial licensing, proprietary compilers, or dedicated paid server infrastructure is required. The project is economically sound and cost-effective.

3.3 High Level Design & Description of Algorithms

3.3.1 Working Mechanism of Proposed System

The working mechanism of the file compression and decompression lifecycle is summarized below:

1. Compression Pipeline:

- Read raw binary data from uploaded file.
- Count frequency of each byte value (0x00 to 0xFF).
- Insert leaf nodes into a min-heap priority queue sorted by frequency.
- Repeatedly extract two nodes with the lowest frequencies, create an internal parent node with their combined frequency, and re-insert into the queue until one root node remains.
- Traverse the tree from root to leaf to assign binary prefix codes (left branch = '0', right branch = '1').
- Encode the original byte sequence using the generated variable-length bit codes.
- Pack bits into 8-bit bytes, prepend the frequency header table, and store the output file ('.shuf').

2. Decompression Pipeline:

- Parse the frequency table header from the ‘.shuf’ file.
- Reconstruct the exact Huffman binary tree.
- Read packed bits sequentially and traverse the tree from root until reaching a leaf node.
- Emit the corresponding original byte and reset traversal to root for the next bits.
- Verify original file size and SHA-256 hash checksum.

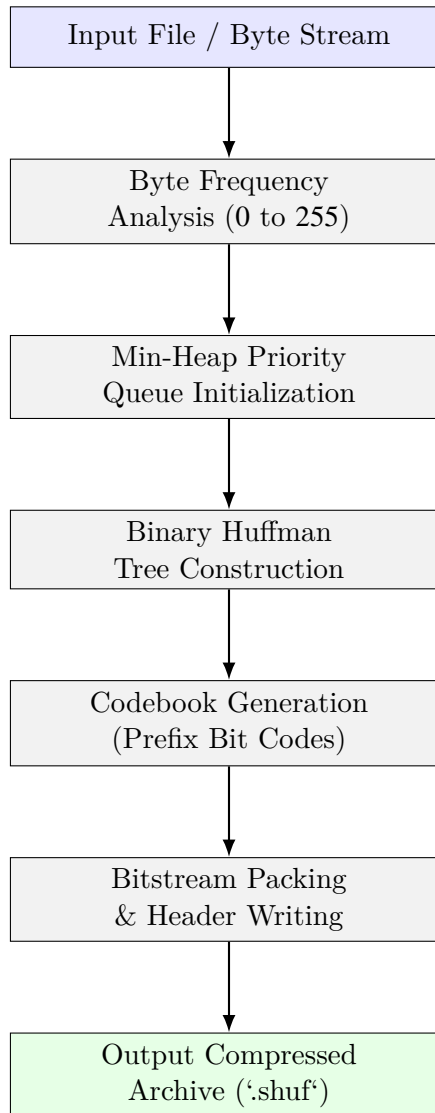


Figure 3.1: High Level Flowchart of Proposed Huffman Compression System

Chapter 4

Project Timeline & Expected Outcomes

4.1 Gantt Chart (Project Timeline)

The proposed project is scheduled for a completion timeline of four (4) months. Table 4.1 illustrates the work breakdown schedule across the four-month implementation lifecycle.

Table 4.1: Four-Month Project Work Schedule (Gantt Chart)

Activity / Milestone	Month 1	Month 2	Month 3	Month 4
Requirement Gathering & Feasibility	[XXXX]			
Literature Review & Algorithmic Study	[XXXX]			
System Architecture & Database Design		[XXXX]		
Huffman Engine Core Implementation		[XXXX]		
Bitstream Packing & Tree Serialization			[XXXX]	
Web Interface & API Integration			[XXXX]	
Lossless Verification & Testing				[XXXX]
Documentation, Report & Defense				[XXXX]

4.2 Expected Outcomes

Upon completion of this project, the following deliverables and outcomes are anticipated:

1. A fully working, lossless file compression and decompression software tool implementing Huffman entropy coding.
2. Documented size reductions ranging from 20% to 60% on text documents, source code, and structured log files.
3. Perfect file restoration confirmed through automated SHA-256 hash comparison.
4. An intuitive, web-based graphical interface for effortless drag-and-drop user interaction.

5. Complete technical documentation and source code repository available for academic evaluation.

Chapter 5

Conclusion & References

5.1 Conclusion

Data compression continues to serve as an indispensable pillar of modern digital communications and storage architecture. This project proposal establishes the technical blueprint for developing an accessible, reliable, and mathematically verified File Compression System grounded in the principles of Huffman Coding.

By focusing strictly on Huffman entropy encoding, the system eliminates unnecessary complexity while delivering optimal prefix code construction, rigorous lossless restoration, and user-friendly interaction. The planned four-month execution schedule ensures orderly progression from theoretical analysis to full implementation, empirical verification, and academic delivery.

5.2 References

1. D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952.
2. C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, July 1948.
3. D. E. Knuth, "Dynamic Huffman Coding," *Journal of Algorithms*, vol. 6, no. 2, pp. 163–180, June 1985.
4. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
5. K. Sayood, *Introduction to Data Compression*, 5th ed. San Francisco, CA: Morgan Kaufmann, 2017.
6. I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic Coding for Data Compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, June 1987.